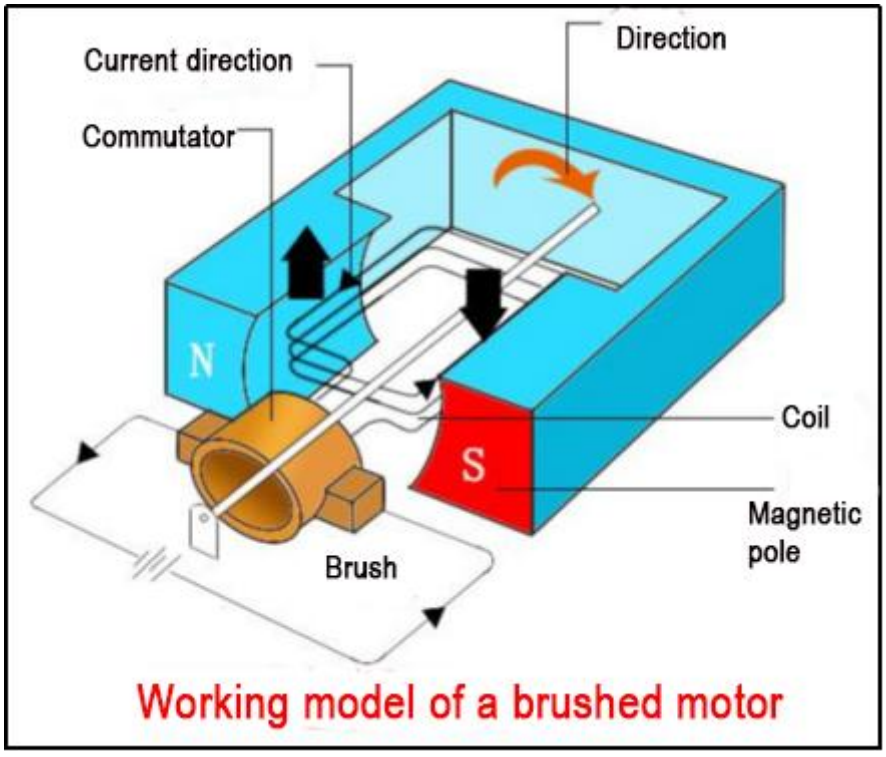
Lesson 9 RRC Brushed DC Motor Program Analysis

1. Preface

The DC motors available in the market are mainly divided into two types: brushless motors and brushed motors. This chapter focuses on explaining brushed DC motors.

Brushed Motors:

In brushed DC motors, current flows through the armature coil, generating a magnetic field that interacts with the magnetic field of the permanent magnet, causing the motor to rotate. The following diagram illustrates the working model of a brushed DC motor.



Components: Permanent magnet, armature coil, brushes, bearings. Speed control principle: By changing the voltage of the power supply, the force on the

rotor coil will also change, thereby changing the motor speed.

Commutation principle: Changing the direction of current in the coil can achieve motor commutation.

Brushless Motors:

Working principle: The stator of a brushless DC motor is a coil winding armature, and the rotor is a permanent magnet. The real-time position of the motor rotor is detected, and then different phase currents are applied to the motor according to the rotor position, so that the stator generates a uniformly changing rotational magnetic field, allowing the motor to follow the magnetic field and rotate.

In the previous section, we learned about the principle of brushed DC motors. In this chapter, we will learn about the source code of brushed DC motors. To do this, we need to prepare STM32CubeIDE or Keil5, and a brushed DC motor.

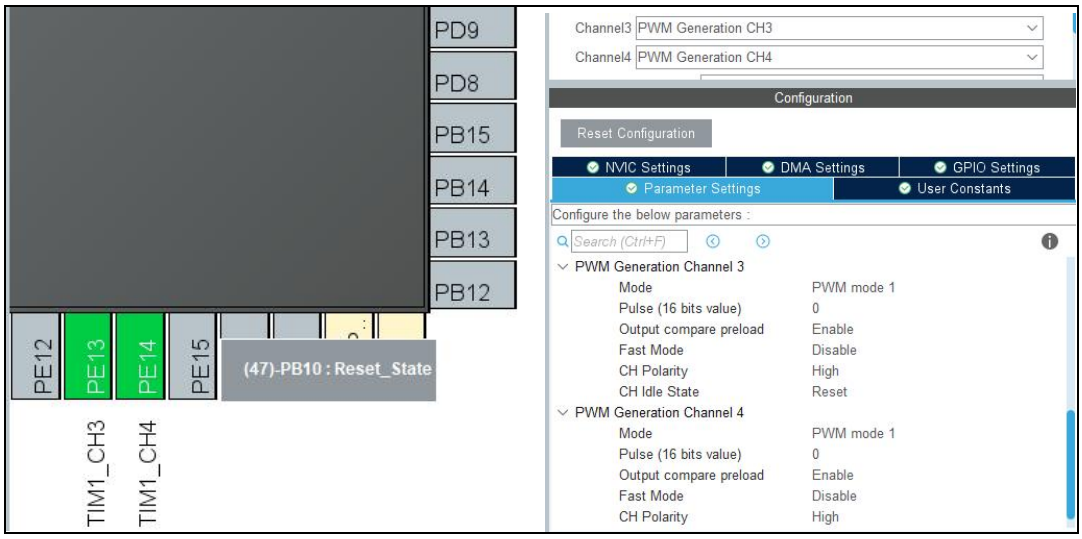
2. Program Storage Path

The example routine for this particular content is named "Motor_Demo":

Program path: Motor_Demo\motor_porting.c

STM32CubeMX path: Motor_Demo\Motor_Demo.ioc

3. Configure the Timer

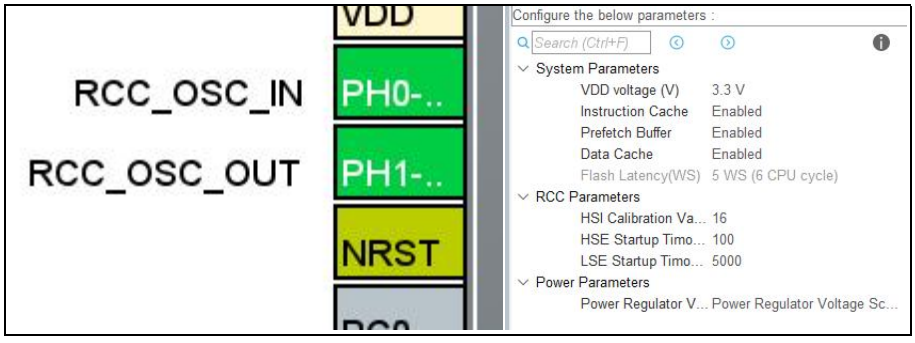


In STM32CubeIDE, open the STM32CubeMX Configurator. In the Peripherals section, locate TIM1 and enable it. Then, in the Configuration tab, set TIM1 for PWM Generation CH3 and CH4. This will be used for generating PWM signals.

Next, in the STM32CubeMX Configurator, go to TIM1 Configuration. In the PWM Generation Channel 3 and PWM Generation Channel 4 sections, set the appropriate PWM frequency and duty cycle. Click OK to save the settings.

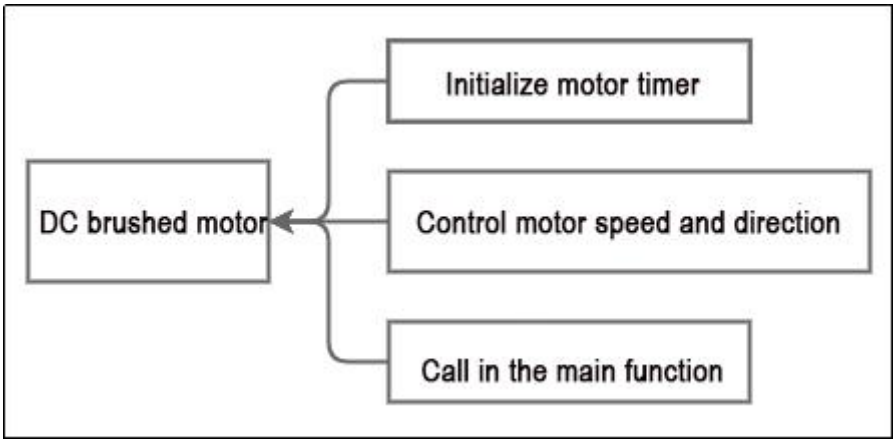
Click on Project > Generate Code to generate the initialization code.

4. Clock configuration



5. Program Source Code Analysis

Program Structure Diagram



5.1 Initialize Motor Timer

```
void motor_init(void)
{
    /* 电机控制 */
    __HAL_TIM_SET_COUNTER(&htim1, 0); /* 清除定时器计数 */
    __HAL_TIM_ENABLE(&htim1);         /* 启动PWM信号生成定时器 */
    __HAL_TIM_MOE_ENABLE(&htim1);     /* 开启PWM信号输出 */
}
```

Reset the counter of the TIM1 timer using the __HAL_TIM_SET_COUNTER() function.

Start the TIM1 timer counting using the __HAL_TIM_ENABLE() function.

Enable the Main Output Enable (MOE) of the TIM1 timer to enable PWM signal output using the __HAL_TIM_MOE_ENABLE() function.

5.2 Control Motor Speed and Direction

```
void motor1_set_pulse(int speed)
{
    if(speed > 0) {  /* 正转 */
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, 0);
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_4, speed);
    } else if(speed < 0) {  /* 反转 */
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_4, 0);
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, -speed);
    } else {  /* 停止 */
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, 0);
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_4, 0);
    }
    HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_3);
    HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_4);
}
```

The function motor1_set_pulse() controls a DC motor by adjusting the duty cycle of the PWM signal, thereby controlling the motor's rotation speed and direction.

If the value of 'speed' is greater than 0, the motor will rotate forward. The function sets the duty cycle of TIM_CHANNEL_3 to 0 and sets the duty cycle of TIM_CHANNEL_4 to 'speed'.

If the value of 'speed' is less than 0, the motor will rotate backward. The function sets the duty cycle of TIM_CHANNEL_4 to 0 and sets the duty cycle of TIM_CHANNEL_3 to the absolute value of 'speed'.

If the value of 'speed' is equal to 0, the motor will stop. The function sets the duty cycle of both TIM_CHANNEL_3 and TIM_CHANNEL_4 to 0.

Finally, the function starts the generation of PWM signals on TIM_CHANNEL_3 and TIM_CHANNEL_4.

5.3 Call in the Main Function

```
while (1) {
    printf("counter:%lld, speed:%0.3fr/s\r\n", motor1.counter, motor1.rps);
    HAL_Delay(100);
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
```

Within the while loop in the main function, there is a call to the motor1 motor structure object declaration. Here, motor1.rps and motor1.counter respectively represent the number of revolutions per second of the motor and the current count value obtained by the motor's counter.

```
/**
 * @brief 编码器电机对象结构体
*/
struct EncoderMotorObject {
    int64_t counter;        /**< @brief 总计数值，64bit 认为不会溢出 */
    int64_t overflow_num;   /**< @brief 溢出计数 */
    int32_t ticks_overflow; /**< @brief 计数溢出值 */
    float tps;              /**< @brief ticks per second 计数器频率 */
    float rps;              /**< @brief revolutions per second 输出轴转速 转每秒 */

    int32_t ticks_per_circle; /**< @brief 电机输出轴旋转一圈产生的计数个数，根据电机实际情况填写 */

};
```

The following is the structure definition for the encoder motor object. You can refer to the comments for detailed information.